OPTIONS ANALYSIS

04

Client Update System Options

Comparison of Content Management Approaches

PREPARED FOR	PREPARED BY	DATE	DOCUMENT
Precision Structures Inc.	EBYG Automation — SSC Digital Services	March 19, 2026	04 of 17

Project: Precision Structures Inc. Website **Prepared by:** SSC Development Team **Date:** March 18, 2026

Purpose

Precision Structures needs a way to request or make updates to their website content. This document compares four approaches, ranging from fully managed (we do all the work) to self-service (the client edits content directly). Each option is evaluated on cost, complexity, risk, and fit for a small manufacturing company with limited technical staff.

Option A: Structured Change Request System (Email/Form-Based)

How It Works

The client sends change requests to the development team via email or a simple web form. Our team implements the changes, deploys to a staging environment for review, and publishes to production after client approval.

Workflow

1. Client identifies a needed change (new photo, updated phone number, revised service description, etc.).
2. Client submits the request via:
   - Email to a designated address (e.g., support@ssc-dev.com), or
   - A simple web form (Google Form, Microsoft Form, or Jotform) with fields for: page, section, description of change, attachments.
3. Our team receives the request, logs it, and implements the change on the `develop` branch.
4. The change deploys automatically to the staging environment.
5. We notify the client that the staging site is ready for review.
6. Client reviews and approves (or requests further changes).
7. On approval, we merge to `master` and the change goes live.

Pros

- **Zero risk of client breaking the site.** All changes go through a developer.
- **No training required.** The client already knows how to send email.
- **No additional software or infrastructure.** The current deployment pipeline is unchanged.
- **Quality control.** Every change is reviewed by a developer before it goes live.
- **Low maintenance.** No admin portal to host, secure, or update.

Cons

- **Turnaround depends on our availability.** Simple changes (a phone number) might take hours instead of minutes.
- **Ongoing cost.** Each change request is billable time (or covered under a retainer).
- **No self-service.** The client cannot make even trivial changes independently.

Cost

ITEM	ESTIMATE
Setup (form, email, process docs)	2-4 hours
Per-change implementation	15-60 min depending on scope
Monthly retainer (optional)	Negotiable, covers N changes/month

Best Fit

This is the best option for Precision Structures today. The site is small (6 pages), changes are infrequent, and the client team is non-technical. This approach carries the lowest risk and the lowest upfront cost.

Option B: Lightweight Admin Portal (Custom-Built)

How It Works

We build a simple, password-protected web application that exposes specific content fields for editing. The client logs in, changes a text block or uploads an image, previews the result on a staging environment, and submits for approval. Locked areas (navigation, layout, CSS, scripts) are not editable.

Workflow

1. Client logs into the admin portal.
2. Selects a page and content area (e.g., "Home > Hero Text").
3. Edits the text or uploads a new image.
4. Clicks "Preview" to see the change on the staging site.
5. Clicks "Submit for Review."
6. Our team receives a notification, reviews the change, and approves or rejects it.
7. On approval, the change is deployed to production.

Editable Content Areas

- Hero section text and images

- About page body text
- Service descriptions
- Gallery images
- Testimonials and quotes
- Contact information (phone, email, address, hours)

Locked Areas (Not Editable by Client)

- Navigation structure and links
- Page layout and HTML structure
- CSS and visual design
- Footer structure
- JavaScript and interactive features

Pros

- **Client independence for simple changes.** Text updates and image swaps can happen without waiting for us.
- **Design-safe.** Only approved content fields are exposed; the client cannot break the layout.
- **Preview before publish.** Changes are visible on staging before going live.
- **Audit trail.** Every edit is logged with who, what, and when.

Cons

- **Custom development required.** The portal must be built, tested, and deployed.
- **Maintenance burden.** If the site structure changes, the portal must be updated to match.
- **Hosting cost.** The portal needs its own hosting (Azure App Service or similar).
- **Authentication and security.** User management, JWT tokens, session handling, etc.

Cost

ITEM	ESTIMATE
Design and development	40-80 hours
Azure hosting (App Service + Blob Storage)	~$20-50/month
Ongoing maintenance	2-4 hours/month
Client training session	1-2 hours

Best Fit

Good upgrade path if the client finds that change requests are too frequent or too slow under Option A. Suitable for clients who make small text and image changes weekly or more often.

A detailed technical specification for this option is available in client-update-portal-spec.md.

Option C: Headless CMS (Sanity, Strapi, or Contentful)

How It Works

A third-party content management system stores structured content (text, images, metadata) in a cloud-hosted backend. The website pulls content from the CMS via API calls or at build time. The client edits content through the CMS's web interface.

Example Platforms

PLATFORM	HOSTING	FREE TIER	NOTES
Sanity	Cloud (hosted)	Yes (generous)	Real-time collaboration
Strapi	Self-hosted	Open source	Full control, more setup
Contentful	Cloud (hosted)	Yes (limited)	Enterprise-oriented

Workflow

1. We define content types in the CMS (e.g., "Service" with fields: title, description, icon, sort order).
2. We modify the website to fetch content from the CMS API (or generate static pages at build time).
3. Client logs into the CMS dashboard, edits content, and publishes.
4. The website updates automatically (via webhook-triggered rebuild or client-side API fetch).

Pros

- **Professional content management.** Rich text editors, image management, content modeling.
- **Scalable.** Easily supports dozens of content types, multiple editors, localization.
- **Collaboration features.** Multiple editors, draft/publish workflows, content scheduling.
- **Vendor-supported.** The CMS platform handles hosting, backups, and security of the content layer.

Cons

- **Adds significant complexity.** The site currently has zero build tools or external dependencies. A CMS introduces an API layer, a build step (or client-side fetching), and a new platform to manage.
- **Requires build tooling changes.** The site would need a static site generator (Eleventy, Astro, etc.) or client-side JavaScript to pull content from the API. This is a meaningful architecture change.
- **Learning curve.** The CMS dashboard is more complex than a simple form. The client must understand content types, publishing states, and media management.
- **Monthly cost.** Free tiers exist but are limited. Paid plans range from $30-300+/month depending on usage.
- **Vendor lock-in.** Content is stored in a third-party system. Migration requires export/import tooling.

Cost

ITEM	ESTIMATE
CMS setup and content modeling	20-40 hours
Website integration (API/build)	30-60 hours
CMS platform cost	$0-99/month (depends on tier)
Ongoing maintenance	2-4 hours/month
Client training	2-4 hours

Best Fit

Best for larger websites with many pages, multiple content editors, and frequent updates. Overkill for a 6-page site with one or two people making occasional changes. Not recommended for Precision Structures at this time.

Option D: Git-Based Editorial (Prose.io, Decap CMS, or Similar)

How It Works

A lightweight editing interface sits on top of the Git repository. When the client edits content through the web UI, the tool commits the changes directly to the repository, which triggers the standard deployment pipeline.

Example Platforms

PLATFORM	HOW IT WORKS	NOTES
Decap CMS	Single-page app added to the site	Formerly Netlify CMS
Prose.io	Web editor for GitHub-hosted files	Markdown-focused
TinaCMS	Git-backed, visual editing	More modern, React-based

Workflow

1. We add the editorial tool to the repository (usually a single config file and a small JS bundle).
2. Client navigates to the editing interface (e.g., `yoursite.com/admin`).
3. Client edits content in a form or markdown editor.
4. On save, the tool commits the change to the `develop` branch.
5. Azure Static Web Apps deploys the change to staging automatically.
6. After review, we merge to `master` for production.

Pros

- **Version control built in.** Every change is a Git commit with full history.
- **No separate backend.** No database, no API server, no additional hosting.
- **Free.** Most Git-based CMS tools are open source with no recurring fees.
- **Familiar deployment.** Uses the same Git workflow the development team already uses.

Cons

- **Technical feel.** The editing interfaces tend to look and feel developer-oriented.
- **Markdown-based.** Most of these tools work best with markdown content, not raw HTML. The site would need to be restructured to use markdown with a static site generator.
- **Limited to text.** Image handling is often clunky (committed as binary files to the repo).
- **Authentication complexity.** Requires GitHub/GitLab OAuth or a proxy backend for authentication.
- **Architecture change.** Like Option C, this likely requires introducing a static site generator.

Cost

ITEM	ESTIMATE
Setup and configuration	15-30 hours
Site restructuring (markdown/templates)	20-40 hours
Ongoing maintenance	1-2 hours/month
Platform cost	$0

Best Fit

Good for developer-adjacent clients who are comfortable with a slightly technical interface. Works well for blog-heavy sites or documentation. Not a natural fit for Precision Structures, where the client team has no development background.

Comparison Summary

CRITERIA	OPTION A (CHANGE REQUESTS)	OPTION B (ADMIN PORTAL)	OPTION C (HEADLESS CMS)	OPTION D (GIT-BASED)
Upfront cost	Very low	Medium	Medium-High	Medium
Ongoing cost	Per-change or retainer	~$20-50/mo hosting	$0-99/mo SaaS	$0
Client independence	None	Partial (safe fields)	Full	Partial
Risk of breaking site	None	Very low	Low	Low-Medium
Technical skill required	None	Minimal	Moderate	Moderate
Setup time	2-4 hours	40-80 hours	50-100 hours	35-70 hours
Maintenance burden	None (us)	Low	Low-Medium	Low
Scalability	Low	Medium	High	Medium

Recommendation

Primary: Option A (Structured Change Request System)

This is the right starting point for Precision Structures. The site is small, updates are infrequent, and the client team is focused on running a truss manufacturing business, not managing a website. A simple email or form-based process gives us full quality control with zero risk to the site.

We recommend setting up:

- A dedicated email address or shared inbox for change requests.
- A simple intake form with fields for page, section, change description, and file attachments.
- A standard turnaround SLA (e.g., minor changes within 1-2 business days, larger changes quoted separately).
- A monthly retainer option that covers a set number of changes.

Upgrade Path: Option B (Lightweight Admin Portal)

If the client finds that they are making frequent changes (weekly or more) and the turnaround time of Option A becomes a pain point, we can build the admin portal described in Option B. This gives the client independence for safe, predefined content areas while keeping the design locked down.

The portal can be built incrementally: start with contact info and text fields, then add image uploads and gallery management over time.

Not Recommended at This Time: Options C and D

Both options introduce significant architectural complexity for a site that does not need it. They are worth revisiting only if the site grows substantially (e.g., adding a blog, product catalog, or multi-user content team).